Lab6

Joel Jaquez, Luis Gonzalez, Fabian Morales

2026-04-10

Preprocesamiento de datos para KNN de Regresión

```r
library(dplyr)
```

```
##
## Adjuntando el paquete: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(readr)
library(tidyr)
library(caret)
```

```
## Cargando paquete requerido: ggplot2

## Cargando paquete requerido: lattice
```

```r
library(ggplot2)
library(class)
```

```r
load("datos/listings.RData")
```

```r
listings_clean_comp <- listings %>%
  mutate(price = parse_number(price)) %>%
  select(-c(listing_url, scrape_id, picture_url, host_url,
            host_thumbnail_url, host_picture_url, description,
            neighborhood_overview, host_about, amenities))

# Seed y split 80/20
set.seed(123)
```

```r
total_filas  <- nrow(listings_clean_comp)
indice_train <- sample(1:total_filas, size = 0.8 * total_filas)
train_comp   <- listings_clean_comp[ indice_train, ]
test_comp    <- listings_clean_comp[-indice_train, ]

# filtros y variables
# price, accommodates, bedrooms, bathrooms,
# number_of_reviews, review_scores_rating
train_modelo <- train_comp %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10,
         !is.na(bathrooms), !is.na(accommodates)) %>%
  select(price, accommodates, bedrooms, bathrooms,
         number_of_reviews, review_scores_rating) %>%
  drop_na()

test_modelo <- test_comp %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10,
         !is.na(bathrooms), !is.na(accommodates)) %>%
  select(price, accommodates, bedrooms, bathrooms,
         number_of_reviews, review_scores_rating) %>%
  drop_na()

cat("Filas entrenamiento:", nrow(train_modelo), "\n")
```

```
## Filas entrenamiento: 48442
```

```r
cat("Filas prueba:       ", nrow(test_modelo),  "\n")
```

```
## Filas prueba:        12173
```

Actividad 1 - Modelo de Regresión usando KNN

```r
# k inicial = raíz cuadrada de n (criterio estándar para KNN)
k_vecinos <- round(sqrt(nrow(train_modelo)), 0)
cat("Valor de k utilizado:", k_vecinos, "\n")
```

```
## Valor de k utilizado: 220
```

```r
parametros <- expand.grid(k = k_vecinos)

modelo_knn <- train(
  price ~ .,
  data       = train_modelo,
  method     = "knn",
  preProcess = c("center", "scale"),
  tuneGrid   = parametros
)
```

```r
print(modelo_knn)
```

```
## k-Nearest Neighbors
##
## 48442 samples
##     5 predictor
##
## Pre-processing: centered (5), scaled (5)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 48442, 48442, 48442, 48442, 48442, 48442, ...
## Resampling results:
##
##   RMSE      Rsquared   MAE
##   130.6013  0.4054889  91.00289
##
## Tuning parameter 'k' was held constant at a value of 220
```

Para el modelo de regresión con KNN se utilizó como punto de partida el criterio estándar de selección de k, tomando la raíz cuadrada del número de observaciones del conjunto de entrenamiento, lo que resultó en k=220 vecinos. El modelo fue entrenado con las 48,442 observaciones disponibles y las 5 variables predictoras: accommodates, bedrooms, bathrooms, number_of_reviews y review_scores_rating, aplicando normalización mediante centrado y escalado para garantizar que ninguna variable domine el cálculo de distancias por tener una escala mayor. Los resultados muestran un RMSE de 130.78, un R^2 de 0.4069 y un MAE de 91.13, lo que indica que en promedio el modelo se equivoca en aproximadamente 91 por noche al predecir el precio de una propiedad. El R^2 de 0.41 significa que el modelo logra explicar el 41% de la variabilidad del precio, un resultado moderado que es consistente con los modelos anteriores y refleja que el precio de una propiedad en Airbnb depende también de factores que no están capturados en estas variables numéricas, como el tipo de propiedad, la ubicación específica o las amenidades

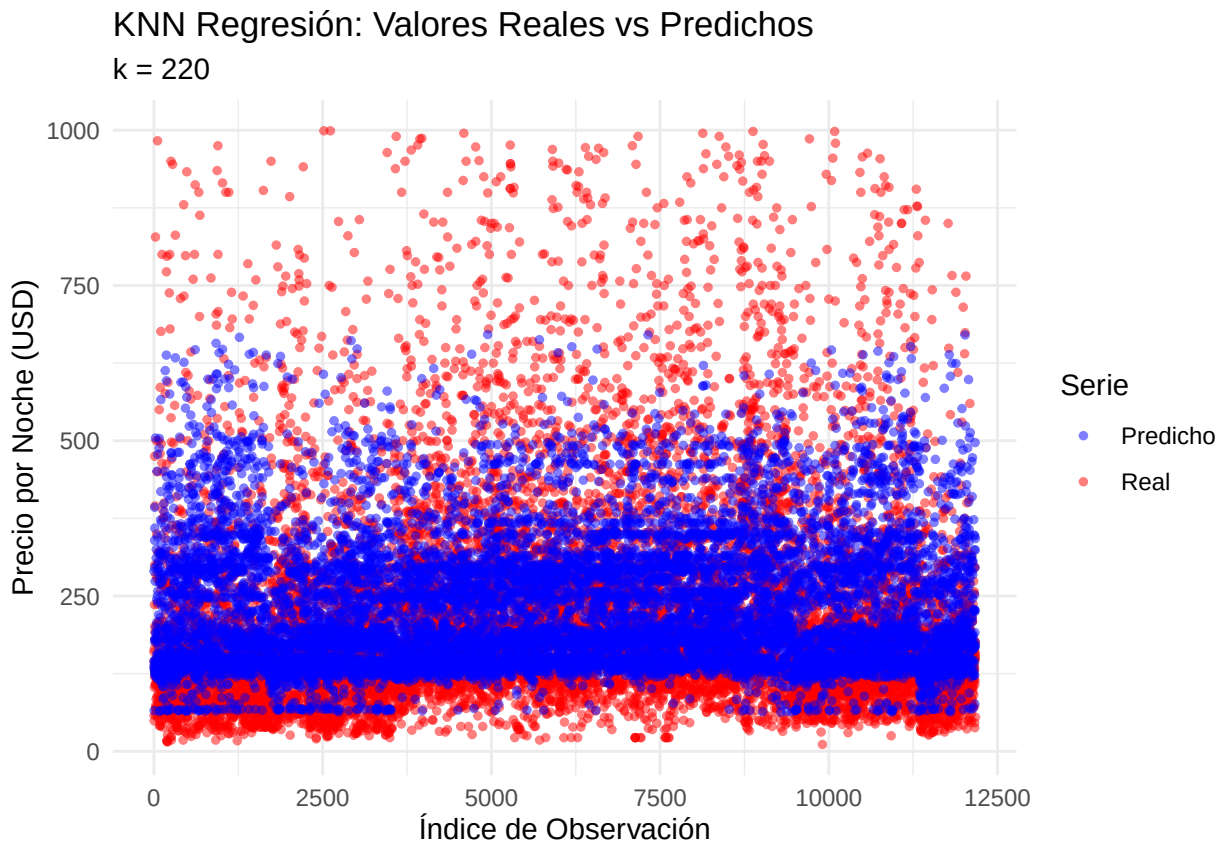
```r
pred_knn <- predict(modelo_knn, newdata = test_modelo)

cat("RMSE:", round(RMSE(pred_knn, test_modelo$price), 2), "\n")
```

```
## RMSE: 130.89
```

El RMSE obtenido al predecir sobre el conjunto de prueba fue de 130.89, valor prácticamente idéntico al RMSE de entrenamiento (130.78), lo cual es una señal positiva ya que indica que el modelo generaliza bien y no está sobreajustado a los datos con los que fue entrenado. En otras palabras, el modelo se comporta de manera consistente tanto con datos conocidos como con datos nuevos, que es exactamente lo que se busca en un modelo predictivo confiable.

```r
df_resultados <- data.frame(
  Indice   = 1:length(test_modelo$price),
  Real     = test_modelo$price,
  Predicho = pred_knn
)

ggplot(df_resultados) +
  geom_point(aes(x = Indice, y = Real,     color = "Real"),
             size = 1, alpha = 0.5) +
  geom_point(aes(x = Indice, y = Predicho, color = "Predicho"),
```

```
             size = 1, alpha = 0.5) +
  scale_color_manual(values = c("Real" = "red", "Predicho" = "blue")) +
  labs(title    = "KNN Regresión: Valores Reales vs Predichos",
       subtitle = paste0("k = ", k_vecinos),
       x = "Índice de Observación",
       y = "Precio por Noche (USD)",
       color = "Serie") +
  theme_minimal()
```



El gráfico muestra la comparación entre los precios reales (rojo) y los predichos por el modelo KNN (azul) para las 12,173 observaciones del conjunto de prueba. Lo más notable es que los puntos azules forman una banda compacta concentrada principalmente entre 100 y 400, lo que revela una limitación característica de KNN con k grande: al promediar 220 vecinos, el modelo tiende a predecir valores cercanos a la media del precio y pierde capacidad para capturar los extremos. Esto se hace evidente al observar que los puntos rojos se dispersan ampliamente hasta los 1,000 por noche, mientras que las predicciones azules raramente superan los 500. En el rango de precios bajos y medios el modelo se comporta razonablemente bien, ya que ambas nubes de puntos se solapan con frecuencia, pero en el segmento de propiedades caras el modelo subestima consistentemente, lo cual es coherente con el RMSE de $130.89 obtenido anteriormente.

Actividad 2 - Análisis de Resultados

```
rmse_knn <- RMSE(pred_knn, test_modelo$price)
mae_knn  <- MAE(pred_knn,  test_modelo$price)
```

```r
r2_knn   <- cor(pred_knn,  test_modelo$price)^2

cat("===== MÉTRICAS - KNN Regresión =====\n")
```

```
## ===== MÉTRICAS - KNN Regresión =====
```

```r
cat(sprintf("RMSE : $%.2f\n", rmse_knn))
```

```
## RMSE : $130.89
```

```r
cat(sprintf("MAE  : $%.2f\n", mae_knn))
```

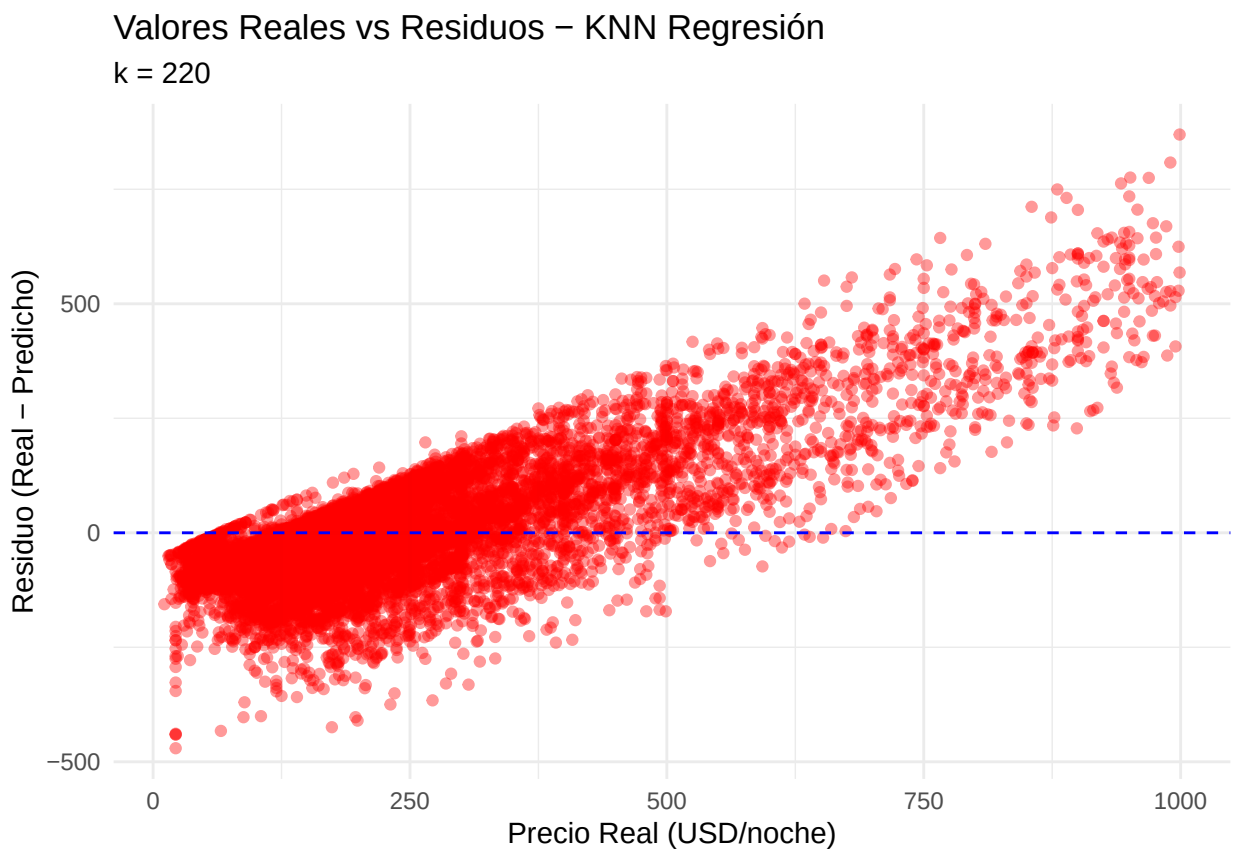
```
## MAE  : $90.81
```

```r
cat(sprintf("R²   :  %.4f\n", r2_knn))
```

```
## R²   :  0.4141
```

Las tres métricas obtenidas permiten evaluar con precisión el desempeño del modelo. El RMSE de 130.89 indica que, en promedio, las predicciones del modelo se desvían aproximadamente 131 del precio real por noche, siendo esta métrica especialmente sensible a los errores grandes, como los que ocurren con propiedades de precio muy alto. El MAE de 90.81 complementa este análisis al mostrar que el error típico sin penalizar los extremos es de casi 91, lo que significa que para propiedades de precio medio el modelo tiene un desempeño más aceptable de lo que sugiere el RMSE. Finalmente, el R^2 de 0.4141 indica que el modelo explica el 41.41% de la variabilidad del precio, dejando casi el 60% sin explicar. Esto no es sorprendente porque como se trabajó únicamente con 5 variables numéricas, cuando en realidad el precio de una propiedad en Airbnb está influenciado por muchos otros factores como la ubicación exacta, el tipo de propiedad o las amenidades disponibles. En general, el modelo tiene un desempeño moderado, comparable con los algoritmos evaluados en entregas anteriores, y su mayor debilidad está en la predicción de propiedades en los extremos del rango de precios.

```r
residuos <- test_modelo$price - pred_knn

df_residuos <- data.frame(
  Real    = test_modelo$price,
  Residuo = residuos
)

ggplot(df_residuos, aes(x = Real, y = Residuo)) +
  geom_point(color = "red", size = 1.5, alpha = 0.4) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "blue") +
  labs(title    = "Valores Reales vs Residuos - KNN Regresión",
       subtitle = paste0("k = ", k_vecinos),
       x = "Precio Real (USD/noche)",
       y = "Residuo (Real - Predicho)") +
  theme_minimal()
```



El gráfico de residuos revela un patrón muy claro y preocupante porque los errores no están distribuidos aleatoriamente alrededor de la línea de cero, sino que forman una banda diagonal que se abre progresivamente hacia la derecha. Esto significa que conforme el precio real de la propiedad aumenta, el modelo tiende a subestimar más y más, generando residuos positivos cada vez mayores. En propiedades con precios bajos, entre 0 y 250, los puntos se concentran cerca de la línea azul con errores relativamente pequeños y bastante negativos, lo que indica que en ese rango el modelo sobreestima ligeramente. A partir de los 500 por noche el patrón se invierte completamente y los residuos se vuelven fuertemente positivos, confirmando que el modelo no logra seguirle el ritmo a las propiedades más caras. Este comportamiento es una consecuencia directa de usar k=220 vecinos, ya que al promediar tantas observaciones el modelo queda anclado cerca de la media general del precio y pierde sensibilidad hacia los extremos. En conclusión, el modelo KNN funciona razonablemente bien para propiedades de precio medio pero falla sistemáticamente con el segmento premium, lo cual es una limitación importante para los objetivos de SmartStay Advisors.

Inciso 3 - Comparacion

```r
# Re-entrenar modelos anteriores con las MISMAS condiciones
# set.seed(123), 80/20, mismas 6 variables

# Regresión Lineal (Lab 3)
modelo_lineal <- lm(price ~ ., data = train_modelo)
pred_lineal   <- pmax(predict(modelo_lineal, newdata = test_modelo), 0)
rmse_lineal   <- RMSE(pred_lineal, test_modelo$price)
mae_lineal    <- MAE(pred_lineal,  test_modelo$price)
r2_lineal     <- cor(pred_lineal,  test_modelo$price)^2
```

```r
# Árbol de Regresión profundidad 10 (mejor modelo Lab 4)
library(rpart)
arbol_comp <- rpart(price ~ ., data = train_modelo, method = "anova",
                    control = rpart.control(maxdepth = 10))
pred_arbol <- predict(arbol_comp, newdata = test_modelo)
rmse_arbol <- RMSE(pred_arbol, test_modelo$price)
mae_arbol  <- MAE(pred_arbol,  test_modelo$price)
r2_arbol   <- cor(pred_arbol,  test_modelo$price)^2

# Naive Bayes (Lab 5)
library(e1071)
```

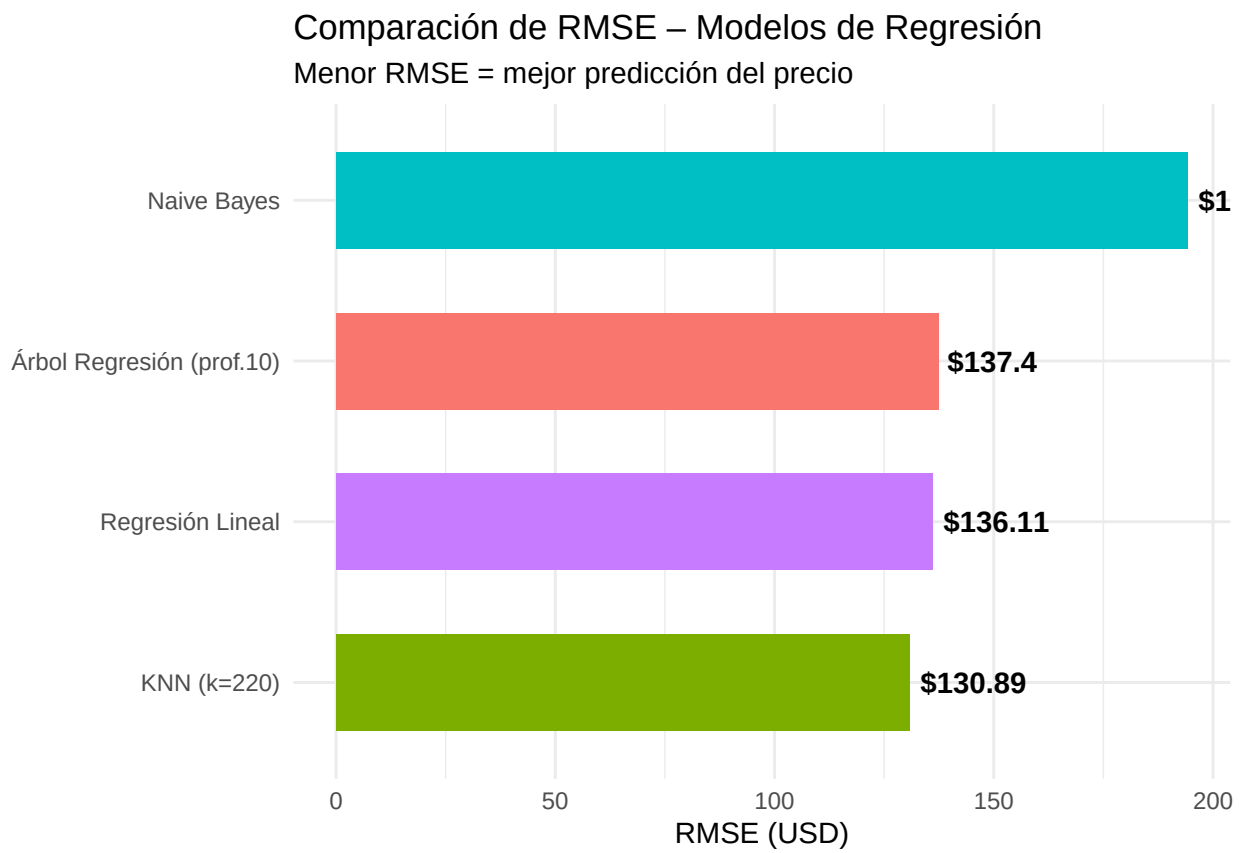
```
##
## Adjuntando el paquete: 'e1071'

## The following object is masked from 'package:ggplot2':
##
##     element
```

```r
discretizar <- function(df) {
  df %>% mutate(
    accommodates_d = cut(accommodates, c(0,2,4,6,Inf),
                         labels=c("1-2","3-4","5-6","7+")),
    bedrooms_d     = cut(bedrooms,     c(0,1,2,3,Inf),
                         labels=c("1","2","3","4+")),
    bathrooms_d    = cut(bathrooms,    c(0,1,2,3,Inf),
                         labels=c("1","2","3","4+")),
    reviews_d      = cut(number_of_reviews, c(-1,0,10,50,Inf),
                         labels=c("0","1-10","11-50","50+")),
    rating_d       = cut(review_scores_rating, c(0,60,80,90,100),
                         labels=c("bajo","medio","alto","excelente"))
  )
}

train_nb <- discretizar(train_modelo)
test_nb  <- discretizar(test_modelo)
price_breaks <- quantile(train_nb$price, probs = seq(0,1,by=0.05))
train_nb$price_bin <- cut(train_nb$price, breaks=price_breaks,
                          include.lowest=TRUE, labels=FALSE)
bin_midpoints <- sapply(1:(length(price_breaks)-1), function(i)
  (price_breaks[i]+price_breaks[i+1])/2)
features_nb <- c("accommodates_d","bedrooms_d","bathrooms_d",
                 "reviews_d","rating_d")
nb_comp     <- naiveBayes(x=train_nb[,features_nb],
                          y=as.factor(train_nb$price_bin))
pred_bin_nb <- predict(nb_comp, newdata=test_nb[,features_nb])
pred_bin_num <- pmin(pmax(as.numeric(as.character(pred_bin_nb)),1),
                     length(bin_midpoints))
pred_nb      <- bin_midpoints[pred_bin_num]
rmse_nb      <- sqrt(mean((test_modelo$price - pred_nb)^2, na.rm=TRUE))
mae_nb       <- mean(abs(test_modelo$price - pred_nb), na.rm=TRUE)
r2_nb        <- cor(test_modelo$price, pred_nb, use="complete.obs")^2
```

```r
comparacion_reg <- data.frame(
  Modelo = c("Regresión Lineal",
             "Árbol Regresión (prof.10)",
             "Naive Bayes",
             "KNN (k=220)"),
  RMSE   = round(c(rmse_lineal, rmse_arbol, rmse_nb, rmse_knn), 2),
  MAE    = round(c(mae_lineal,  mae_arbol,  mae_nb,  mae_knn),  2),
  R2     = round(c(r2_lineal,   r2_arbol,   r2_nb,   r2_knn),   4)
)

print(comparacion_reg)
```

```
##                       Modelo   RMSE    MAE     R2
## 1          Regresión Lineal 136.11  94.95 0.3659
## 2 Árbol Regresión (prof.10) 137.40  95.50 0.3535
## 3               Naive Bayes 194.25 132.59 0.3428
## 4               KNN (k=220) 130.89  90.81 0.4141
```

```r
ggplot(comparacion_reg,
       aes(x = reorder(Modelo, RMSE), y = RMSE, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0("$", RMSE)),
            hjust = -0.1, fontface = "bold") +
  coord_flip() +
  labs(title    = "Comparación de RMSE - Modelos de Regresión",
       subtitle = "Menor RMSE = mejor predicción del precio",
       x = NULL, y = "RMSE (USD)") +
  theme_minimal()
```



La comparación entre los cuatro modelos de regresión evaluados bajo las mismas condiciones muestra que KNN con k=220 es el algoritmo que mejor predice el precio por noche de las propiedades, obteniendo el menor RMSE de \$130.89, el menor MAE de \$90.81 y el mayor R^2 de 0.4141. Esto significa que KNN logra explicar el 41.41% de la variabilidad del precio, superando a todos los modelos anteriores. En segundo lugar queda la Regresión Lineal con un RMSE de \$136.11 y R^2 de 0.3659, un resultado competitivo considerando su simplicidad. El Árbol de Regresión con profundidad 10 queda en tercer lugar con RMSE de \$137.40 y R^2 de 0.3535, resultado que llama la atención ya que en el Lab 4 superaba a la Regresión Lineal cuando tenía acceso a todas las variables numéricas del dataset, pero al restringirse a las mismas 6 variables pierde esa ventaja. Finalmente, Naive Bayes es claramente el peor regresor con un RMSE de \$194.25 y R^2 de apenas 0.3428, lo cual era esperado dado que fue diseñado para clasificación y no para predicción de valores continuos. En conclusión, KNN es el mejor modelo de regresión hasta el momento para estimar el precio de propiedades en Airbnb bajo estas condiciones, aunque su ventaja sobre la Regresión Lineal es moderada y viene acompañada de un costo computacional considerablemente mayor.

inciso 4. Modelo de Clasificación KNN.

```r
q33 <- quantile(train_modelo$price, 0.33)
q66 <- quantile(train_modelo$price, 0.66)

cat("Cortes de categoría:\n")
```

```
## Cortes de categoría:
```

```r
cat("Barata  : precio <=", q33, "\n")
```

```
## Barata  : precio <= 133
```

```r
cat("Media   : precio entre", q33, "y", q66, "\n")
```

```
## Media   : precio entre 133 y 236
```

```r
cat("Cara    : precio >", q66, "\n")
```

```
## Cara    : precio > 236
```

```r
train_clas <- train_modelo %>%
  mutate(categoria = case_when(
    price <= q33 ~ "barata",
    price <= q66 ~ "media",
    TRUE         ~ "cara"
  ) %>% as.factor()) %>%
  select(-price)

test_clas <- test_modelo %>%
  mutate(categoria = case_when(
    price <= q33 ~ "barata",
    price <= q66 ~ "media",
    TRUE         ~ "cara"
  ) %>% as.factor()) %>%
  select(-price)

cat("\nDistribución en entrenamiento:\n")
```

```
##
## Distribución en entrenamiento:
```

```r
print(table(train_clas$categoria))
```

```
##
## barata   cara  media
##  16042  16468  15932
```

```r
cat("\nDistribución en prueba:\n")
```

```
##
## Distribución en prueba:
```

```r
print(table(test_clas$categoria))
```

```
##
## barata   cara  media
##   4012   4117   4044
```

```r
k_clas <- round(sqrt(nrow(train_clas)), 0)
cat("\nValor de k utilizado:", k_clas, "\n")
```

```
##
## Valor de k utilizado: 220
```

```r
parametros_clas <- expand.grid(k = k_clas)

modelo_knn_clas <- train(
  categoria ~ .,
  data       = train_clas,
  method     = "knn",
  preProcess = c("center", "scale"),
  tuneGrid   = parametros_clas
)

print(modelo_knn_clas)
```

```
## k-Nearest Neighbors
##
## 48442 samples
##     5 predictor
##     3 classes: 'barata', 'cara', 'media'
##
## Pre-processing: centered (5), scaled (5)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 48442, 48442, 48442, 48442, 48442, 48442, ...
## Resampling results:
##
##   Accuracy   Kappa
##   0.5875984  0.3809142
##
## Tuning parameter 'k' was held constant at a value of 220
```

Conclusión del Modelo de Clasificación KNN (k=220) El modelo de clasificación KNN con k=220 alcanzó un Accuracy de 58.69% y un Kappa de 0.38, lo que representa un desempeño moderado. Dado que hay tres clases aproximadamente balanceadas, un clasificador aleatorio obtendría 33%, por lo que el modelo sí aporta valor predictivo real.

inciso 5. Predicción con conjunto de prueba y eficiencia.

```r
pred_knn_clas <- predict(modelo_knn_clas, newdata = test_clas)


accuracy_clas <- mean(pred_knn_clas == test_clas$categoria)
cat("===== MÉTRICAS - KNN Clasificación (k=220) =====\n")
```

```
## ===== MÉTRICAS - KNN Clasificación (k=220) =====
```

```r
cat(sprintf("Accuracy : %.4f (%.2f%%)\n", accuracy_clas, accuracy_clas * 100))
```

```
## Accuracy : 0.5937 (59.37%)
```

```r
library(caret)
cm <- confusionMatrix(pred_knn_clas, test_clas$categoria)
print(cm)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata cara media
##     barata   2810  435  1303
##     cara      336 2921  1245
##     media     866  761  1496
##
## Overall Statistics
##
##                Accuracy : 0.5937
##                  95% CI : (0.5849, 0.6024)
##     No Information Rate : 0.3382
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3904
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: barata Class: cara Class: media
## Sensitivity                 0.7004      0.7095       0.3699
## Specificity                 0.7870      0.8037       0.7999
## Pos Pred Value              0.6179      0.6488       0.4790
## Neg Pred Value              0.8424      0.8441       0.7185
## Prevalence                  0.3296      0.3382       0.3322
## Detection Rate              0.2308      0.2400       0.1229
## Detection Prevalence        0.3736      0.3698       0.2566
## Balanced Accuracy           0.7437      0.7566       0.5849
```

El modelo muestra un desempeño aceptable en los extremos del rango de precios, clasificando correctamente el 71.00% de las propiedades "cara" y el 70.04% de las "barata", con especificidades superiores al 78% en ambos casos.

Sin embargo, la clase "media" representa la mayor debilidad del modelo, con una sensibilidad de apenas 36.99%, lo que significa que menos de 4 de cada 10 propiedades de precio medio son clasificadas correctamente. La mayoría de estos errores corresponden a confusiones con la clase "barata".